

Documento de Casos de Teste — E-commerce

Módulo: tests/test_ecommerce.py
Arquivo de testes: tests/test_ecommerce.py
Framework: pytest

Funções testadas

Função	Descrição
Verificar se Banco de Dados Vazio	Retorna se o banco de dados de produtos está vazio
Inserção Produto A	Retorna a inserção de um Produto A no banco de dados
Registro Primeiro Trimestre 2026	Retorna o registro do evento Primeiro Trimestre 2026 na tabela de auditoria
Contagem de Eventos	Retorna a contagem de eventos na tabela de auditoria

Casos de Teste

CT-01 — Soma de números positivos

Campo	Detalhe
ID	CT-01
Função testada	test_banco_produtos_vazio
Objetivo	Verificar se o Banco de Dados de produtos está vazio
Entradas	SELECT COUNT(*) FROM produtos
Resultado esperado	0
Técnica	Igualdade direta (assert ==)
Tipo	Caso normal (caminho feliz)

```
def test_banco_produtos_vazio(db_produtos):  
    cursor = db_produtos.cursor()  
    cursor.execute('SELECT COUNT(*) FROM produtos')  
    total = cursor.fetchone()[0]  
    assert total == 0
```

CT-02 — Inserir Produto na tabela Produtos

Campo	Detalhe
ID	CT-02
Função testada	test_inserir_produto
Objetivo	Verificar se o produto é inserido corretamente na tabela de produtos
Entradas	nome = 'Produto A', preco = 10.0
Resultado esperado	'Produto A'
Técnica	Igualdade direta (assert ==)
Tipo	Caso normal (caminho feliz)

```
def test_inserir_produto(db_produtos):  
    cursor = db_produtos.cursor()  
    cursor.execute(  
        'INSERT INTO produtos (nome, preco) VALUES (?, ?)',  
        ('Produto A', 10.0)  
    )  
    db_produtos.commit()  
  
    cursor.execute('SELECT nome FROM produtos WHERE preco = ?', (10.0,))  
    row = cursor.fetchone()  
  
    assert row[0] == 'Produto A'
```

CT-03 — Registro de evento na Tabela Auditoria

Campo	Detalhe
ID	CT-03
Função testada	test_registrar_log
Objetivo	Verificar se o evento é registrado corretamente na tabela de auditoria
Entradas	Primeiro Trimestre 2026
Resultado esperado	Primeiro Trimestre 2026
Técnica	Igualdade direta (assert ==)
Tipo	Caso normal (caminho feliz)

```
def test_registrar_log(db_session):
    db_session.execute('INSERT INTO auditoria (evento) VALUES (?)', ('Primeiro Trimestre 2026',))
    db_session.commit()

    row = db_session.execute("SELECT evento FROM auditoria WHERE id = '1'").fetchone()
    assert row[0] == 'Primeiro Trimestre 2026'
```



CT-04 — Contagem de Eventos na Tabela Auditoria

Campo	Detalhe
ID	CT-04
Função testada	test_contar_logs
Objetivo	Verificar a contagem de eventos na tabela de auditoria
Entradas	SELECT COUNT(*) FROM auditoria
Resultado esperado	1
Técnica	Igualdade ou maior direta (assert >=)
Tipo	Caso normal (caminho feliz)

```
def test_contar_logs():
    count = db_session.execute('SELECT COUNT(*) FROM auditoria').fetchone()[0]

    assert count >= 1
```

Resumo das técnicas utilizadas

Técnica	Quando usar
assert total == 0	Comparação de valores exatos (inteiros, strings, booleanos)

Como executar os testes

A partir da raiz do projeto (meu-projeto-pytest/):

Rodar todos os testes

pytest

Rodar com detalhes de cada caso

pytest -v

Rodar um teste específico

pytest tests/test_operacoes.py::test_dividir_por_zero_levanta_excecao

Resultado esperado ao rodar pytest -v

```
test_ecommerce.py::test_banco_produtos_vazio PASSED
test_ecommerce.py::test_inserir_produto PASSED
test_ecommerce.py::test_registrar_log PASSED
test_ecommerce.py::test_contar_logs PASSED
```

```
7 passed in 0.07s
```